



基于 SSM 的社区医院管理系统 设计与实现

• 医疗信息化项目实战 •

23级计算机科学与技术02班

组别：四组



PART 01

绪论

University academic reporting graduation reply template



项目背景

- 业务数据分散，流程具有先后顺序
- 社区门诊业务流程多，角色权限复杂

功能目标

- 完成多个角色之间的交互
- 通过数据库存储信息

项目实施

传统门诊流程中，患者挂号后需要医生接诊，医生诊断后可能产生挂号费和药品费，患者缴费后药剂师才能发药。如果这些环节都靠人工记录，容易出现信息不同步、权限混乱、收费状态不清楚等问题。因此本项目把医院业务拆分成四类角色和多个数据表，通过系统完成流程管理。

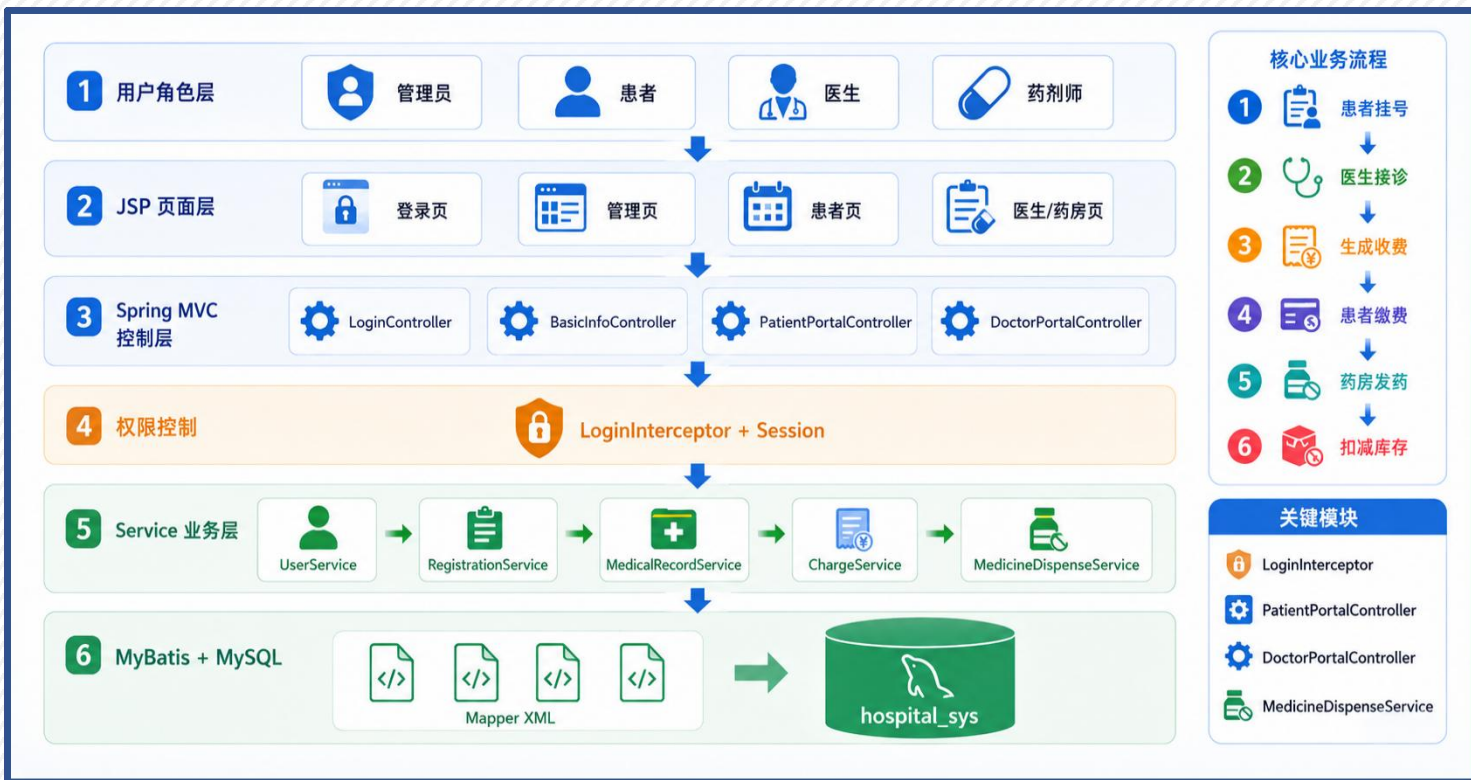


PART 02

项目结构与功能开发

University academic reporting graduation reply template

开发顺序：先搭架构，再打通业务闭环



核心逻辑

技术栈: Spring MVC、Spring、MyBatis、JSP、MySQL、Maven、Tomcat

功能角色: 管理员、患者、药剂师、社区医生

开发顺序: 首先配置项目依赖和 SSM 框架，让请求可以进入 Controller；然后设计数据库表；接着实现登录注册和角色权限；最后围绕门诊业务数据流转。

“项目按业务依赖顺序逐步开发，保证各功能的实现和逻辑闭环。”

```
<servlet>
  <servlet-name>dispatcherServlet</servlet-name>
  <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
  <init-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>classpath:spring-mvc.xml</param-value>
  </init-param>
  <load-on-startup>1</load-on-startup>
</servlet>

<servlet-mapping>
  <servlet-name>dispatcherServlet</servlet-name>
  <url-pattern>/</url-pattern>
</servlet-mapping>
```



```
<context:component-scan base-package="hospital.controller"/>

<mvc:annotation-driven/>
```



```
<bean id="sqlSessionFactory" class="org.mybatis.spring.SqlSessionFactoryBean">
  <property name="dataSource" ref="dataSource"/>
  <property name="configLocation" value="classpath:mybatis-config.xml"/>
  <property name="mapperLocations" value="classpath:mapper/*.xml"/>
</bean>

<bean class="org.mybatis.spring.mapper.MapperScannerConfigurer">
  <property name="basePackage" value="hospital.mapper"/>
  <property name="sqlSessionFactoryBeanName" value="sqlSessionFactory"/>
</bean>
```

webapp/WEB-INF/web.xml

配置 DispatcherServlet,
作为 Spring MVC 请求入口

resources/spring-mvc.xml

扫描 Controller, 并开启
注解驱动。

applicationContext.xml

配置 MyBatis 的
'SqlSessionFactoryBean'
和 Mapper 扫描。

三层职责:

- 接收请求, 返回页面
- 处理业务逻辑
- 执行, 访问数据库

整体框架

浏览器访问 JSP 后, 请求先
进入 web.xml 配置的
DispatcherServlet, 再由
Spring MVC 匹配对应
Controller。

Controller 调用 Service 处
理业务逻辑, 如注册、挂号、收
费等; 数据库操作由 Mapper 接
口和 Mapper XML 绑定后, 通过
MyBatis 执行 SQL。

JSP 页面 -> Spring MVC Controller -> Service -> MyBatis Mapper -> MySQL

持久层设计：用 MyBatis 管理 SQL

数据库：hospital_sys
连接配置：jdbc.properties
数据源：Druid
SQL 映射：mapper/*.xml

配置文件提供连接参数，Spring 创建数据库连接池，MyBatis XML 根据页面条件生成 SQL，最终查询 MySQL 数据库。

jdbc.properties

```
1 jdbc.driver=com.mysql.cj.jdbc.Driver ✓
2 jdbc.url=jdbc:mysql://localhost:3306
  /hospital_sys?useUnicode=true
  &characterEncoding=UTF-8&serverTimezone
  =Asia/Shanghai
3 jdbc.username=root
4 jdbc.password=123456
5 jdbc.initialSize=1
6 jdbc.minIdle=1
7 jdbc.maxActive=10
8 jdbc.maxWait=60000
```

数据库驱动、URL、用户名、密码

applicationContext.xml

```
15 <bean id="dataSource" class="com.alibaba.druid
   .pool.DruidDataSource" init-method="init"
   destroy-method="close">
16   <property name="driverClassName" value="${jdbc
   .driver}" />
17   <property name="url" value="${jdbc.url}" />
18   <property name="username" value="${jdbc
   .username}" />
19   <property name="password" value="${jdbc
   .password}" />
20   <property name="initialSize" value="${jdbc
   .initialSize}" />
21   <property name="minIdle" value="${jdbc
   .minIdle}" />
22   <property name="maxActive" value="${jdbc
   .maxActive}" />
23   <property name="maxWait" value="${jdbc
   .maxWait}" />
```

创建 Druid 数据源，注入对象

PatientMapper.xml

```
17 <select id="findAll" resultType="Patient">
18   SELECT id, name, gender, age, phone, address
19   FROM patients
20   <where>
21     <if test="keyword != null and keyword !=
22       ''">
23       name LIKE CONCAT('%', #{keyword}, '%')
24     </if>
25   </where>
26   ORDER BY id DESC
27 </select>
```

患者列表和关键字查询功能实现

(以患者查询为例)

注册功能实现

```
36 @RequestMapping(value = "/register", method =  
37 RequestMethod.POST) @Log  
38 public String register(HttpServletRequest req) {  
39     String username = trim(req.getParameter  
40     (s: "username"));  
41     String password = trim(req.getParameter  
42     (s: "password"));  
43     String confirmPassword = trim(req  
44     .getParameter(s: "confirmPassword"));  
45     String realName = trim(req.getParameter  
46     (s: "realName"));
```

接收注册POST请求

从页面表单中读取
用户名、密码、确
认密码和真实姓名

非空校验：返回注册页面；调用
userService.findByUsername()判
断账号是否已经存在。校验通过创建
对象，调用注册方法，进入登录页面

注册流程：读取表单 -> 非空校验 -> 密码一致校验 -> 账号重复校验 -> 注册患者

登录流程：读取账号密码 -> 查询用户 -> 保存 Session -> 跳转主页

登录功能实现

```
66 @RequestMapping(value = "/login", method =  
67 RequestMethod.POST) @Log  
68 public String login(HttpServletRequest req) {  
69     String username = req.getParameter  
70     (s: "username");  
71     String password = req.getParameter  
72     (s: "password");  
73  
74     if (isBlank(username) || isBlank(password)) {  
75         req.setAttribute(s: "error", o: "账号和密码  
76         不能为空");  
77         return "login";  
78     }  
79  
80     userService.ensureDefaultUsers();  
81     User user = userService  
82     .findByUsernameAndPassword(username.trim(),  
83     password);  
84     if (user != null) {  
85         req.getSession().setAttribute  
86         (s: "loginUser", user);  
87         return "redirect:/main.jsp";  
88     }
```

读取账号和密码

判断是否为空

确认默认账号存在
保存对象到
loginUser，跳
转页面到系统主
页

loginUser 是后面权限
拦截和业务识别当前用户身
份的基础。

角色初始化与角色权限控制

impl/UserServiceImpl.java

```
44 @Override 1 个用法 15982
45 @Transactional
46 public int registerPatient(User user) {
47     user.setRole("patient");
48     return userMapper.insert(user);
49 }
```

注册患者时设置角色为 patient，避免普通用户直接注册为其他身份

```
17 public boolean preHandle(HttpServletRequest req, HttpServletResponse resp, Object handler)
    throws Exception {
18     String path = req.getRequestURI().substring(
        req.getContextPath().length());
19
20     if (isPublicPath(path)) {
21         return true;
22     }
```

preHandle()方法会在请求进入之前执行。先获取当前请求路径，之后调用isPublicPath(path)，判断放行。

权限控制：不同角色只能访问对应页面

请求路径 -> 是否公共路径 -> 是否登录 -> 判断角色 -> 放行或跳转无权限页面

管理员：基础数据管理

患者：挂号、病历、缴费

医生：挂号表、病历

药剂师：处方发药

判断Session是否为空

```
HttpSession session = req.getSession( false);
if (session == null || session.getAttribute( "loginUser" ) == null) {
    resp.sendRedirect( req.getContextPath() + "/Login.jsp");
    return false;
}
```

不是公共路径

进行登录校验，判断是否继续进入 Controller

取出登录用户，判断管理员/医生/患者/药剂师路径并放行。

之后定义公共路径/管理员路径/医生路径等角色路径。

把权限规则集中在拦截器里统一处理，避免每个 Controller 都重复写角色判断。

基础信息维护（以科室管理为例）

controller/DepartmentController.java

```
20 @RequestMapping(method = RequestMethod.GET)
21 public String list(HttpServletRequest req) {
22     String action = req.getParameter("action");
23
24     if ("new".equals(action)) {
25         return "department-form";
26     }
27
28     if ("edit".equals(action)) {
29         return showEditForm(req);
30     }
31
32     if ("delete".equals(action)) {
33         departmentService.deleteById(parseId(req.getParameter("id")));
34         return "redirect:/departments";
35     }
36
37     return listDepartments(req);
38 }
```

GET 请求进入 list()方法，读取并判断action参数，进行新增/编辑/删除



```
40 @RequestMapping(method = RequestMethod.POST)
41 public String save(HttpServletRequest req) {
42     String name = req.getParameter("name");
43     String description = req.getParameter("description");
44
45     if (name == null || name.trim().isEmpty()) {
46         req.setAttribute("error", "科室名称不能为空");
47         return "department-form";
48     }
49
50     Department department = readDepartment(name, description);
51     if ("update".equals(req.getParameter("action"))) {
52         department.setId(parseId(req.getParameter("id")));
53         departmentService.update(department);
54     } else {
55         departmentService.insert(department);
56     }
57
58     return "redirect:/departments";
59 }
```

读取科室名称和描述，进行非空验证，封装对象，判断action参数。

科室管理：新增、编辑、删除、列表

医生管理：维护姓名、科室、职称、电话、邮箱

患者管理：维护姓名、性别、年龄、电话、地址

药剂师管理：维护基本信息

其他功能的实现与科室管理类似，都是进行通用CRUD操作。

通用 CRUD 模式：

Controller 根据 action 分发操作，Service 承接业务，Mapper XML 执行 SQL。

患者在线预约挂号

页面初始化

```
43 @RequestMapping(value = "/patient-appointment", method = RequestMethod
    .GET) @ 15982
44 public String appointment(HttpServletRequest req) {
45     req.setAttribute("patientName", currentUser
        (req));
46     Registration registration = new Registration();
47     registration.setFee(systemConfigService.getRegistrationFee());
48     req.setAttribute("registration", registration);
49     setOptions(req);
50     return "patient-appointment";
51 }
```

controller/PatientPortalController.java

接收打开预约页面的 GET 请求

设置当前患者姓名

创建Registration对象

读取系统配置中的挂号费

返回预约页面

调用setOptions(req)加载
科室和医生列表

把挂号对象放到 request 中

提交保存

```
if (registration.getDepartmentName() == null || registration
    .getDepartmentName().isEmpty()) {
    req.setAttribute("error", "科室不能为空");
    req.setAttribute("registration", registration);
    req.setAttribute("patientName", patientName);
    setOptions(req);
    return "patient-appointment";
}
```

```
if (!isDoctorInDepartment(registration.getDoctorName(), registration
    .getDepartmentName())) {
    req.setAttribute("error", "医生不属于所选科室");
    req.setAttribute("registration", registration);
    req.setAttribute("patientName", patientName);
    setOptions(req);
    return "patient-appointment";
}

registrationService.insert(registration);
return "redirect:/patient-registrations";
```

该功能对应 SQL 在
RegistrationMapper.
xml, 向 registrations
表插入患者、科室、医
生、费用和状态。

调用isDoctorInDepartment()判断医生是否属于所选科室（挂号逻辑同）

医生接诊步骤：

本人挂号->打开病历页面->提交病历

controller/DoctorPortalController.java

```
44 @RequestMapping(value = "/doctor-registrations", method =  
    RequestMethod.GET) 15982  
45 public String registrations(HttpServletRequest req) {  
46     String doctorName = currentDoctorName(currentUser(req));  
47     req.setAttribute("registrations", registrationService  
        .findByDoctorName(doctorName));  
48     return "doctor-registration-list";  
49 }
```

医生查看本人挂号：

接收请求->获取医生姓名->查询挂号记录->返回挂号列表



```
65 @RequestMapping(value = "/doctor-record-new", method = RequestMethod  
    .GET) 15982  
66 public String newRecord(HttpServletRequest req) {  
67     String doctorName = currentDoctorName(currentUser(req));  
68     req.setAttribute("doctorName", doctorName);  
69     req.setAttribute("patientName", trim(req.getParameter  
        ("patientName")));  
70     req.setAttribute("registrationId", trim(req.getParameter  
        ("registrationId")));  
71     setMedicines(req);  
72     return "doctor-record-form";  
73 }
```

打开病历填写页面：

接收请求->获取医生姓名->信息加入request->加载药品列表->返回挂号列表



提交病历

创建MedicalRecord对象，并设置患者姓名、医生姓名、诊断内容和治疗方案->校验患者姓名不能为空，如果为空就返回表单页面。

病历最终通过recordService.insert(record)保存。也就是说，医生接诊的核心数据是MedicalRecord，它记录了患者、医生、诊断和治疗方案。

业务联动：病历、收费、处方、挂号状态同步更新

保存病历 -> 生成挂号费 -> 判断是否开药 -> 生成处方 -> 计算药费 -> 生成药费 -> 更新挂号状态

controller/DoctorPortalController.java

recordService.insert(record);
完成病历保存



```
128 Integer registrationId = parseOptionalId(req.getParameter
    (s: "registrationId"));
129 if (registrationId != null) {
130     Registration registration = registrationService.findById
        (registrationId);
131     if (registration != null && registration.getFee() != null) {
132         insertCharge(record.getPatientName(), itemName: "挂号费",
            registration.getFee());
133     }
134 }
```

读取，判断非空，生成挂号费收费记录

判断医生是否选择药品。如果选择了药品，调用
dispenseService.prescribe()生成待发放处方，计算药品费用，再次调用
insertCharge()生成药品收费记录。

之后判断挂号 ID 是否存在，如果挂号ID存在，调用
registrationService.updateStatus()把挂号状态更新为已就诊。

```
if (medicine != null) {
    dispenseService.prescribe(record.getPatientName(), medicine, quantity);
    BigDecimal amount = medicine.getPrice() == null ? BigDecimal.ZERO : medicine.getPrice().multiply(new BigDecimal(quantity));
    insertCharge(record.getPatientName(), medicine.getName(), amount);
}
if (registrationId != null) {
    registrationService.updateStatus(registrationId, FINISHED_STATUS);
}
return "redirect:/doctor-records";
```



最后实现insertCharge()，插入更新收费表

医生提交一次病历，系统同时保存病历、生成收费、生成处方，并更新挂号状态。





PART 02

项目过程问题解决

University academic reporting graduation reply template

SSM 迁移后静态资源和请求入口被拦截的问题



多角色系统只隐藏菜单不够，用户可能 URL 越权访问的问题



药品发放涉及缴费状态和库存扣减，容易数据不一致的问题





PART 03

项目成果展示

University academic reporting graduation reply template



基于 SSM 的社区医院管理系统 设计与实现

• 医疗信息化项目实战 •

23级计算机科学与技术02班

组别：四组